Evolution of the BIRDI research pipeline

From latent structural learning to explicit plans and executable verification

1. Graph-latent Python

AST + state/data-flow graph latent

code graph -> JEPA alignment -> graph decoder -> Python

WHAT WE RETAINED

Structural supervision and contrastive alignment can organize problem and solution representations.

WHY WE EVOLVED

The decoder drifted semantically; small prompt changes could produce the same incorrect logic.

2. Typed IR + retrieval

Compact typed algorithm DSL / IR

JEPA latent -> top-k IR -> CodeT5+ repair -> compiler

WHAT WE RETAINED

An explicit target made retrieval, repair, and final code generation separately diagnosable.

WHY WE EVOLVED

Algorithm composition remained limited when prototypes and repair training lacked the needed variation.

3. Verified RTL agents

Verilog + executable oracle evidence

RTL -> generated tests -> Icarus -> repair -> benchmark

WHAT WE RETAINED

Compilation, simulation, mismatch traces, and repair connect model output to measurable behavior.

WHY WE EVOLVED

Random stimuli may miss rare states, and two agents can still agree on the same wrong behavior.

Design principle carried forward

We progressively reduced hidden inference: latent graph -> explicit typed IR -> executable behavioral evidence.

Current implementation: verify and repair before final scoring

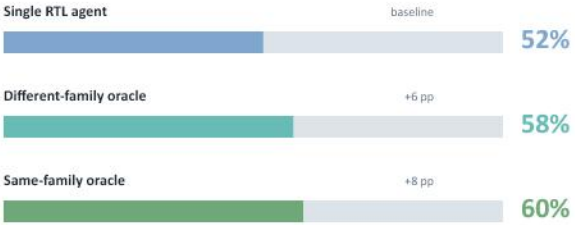
The same RTL generator is evaluated under two oracle-family conditions

Executable multi-agent RTL workflow



Preliminary VerilogEval pass@1 | 50 complexity-weighted tasks

Verification and repair produced the largest observed gain.



Preliminary subset result; denominator = 50 attempted tasks.

Interpretation and next experiment

1. **Verification adds measurable value**
   Both oracle variants improved the same generator's final pass@1.

2. **Family diversity was not automatically superior**
   Qwen oracle reached 60% vs DeepSeek at 58%; capability mattered more in this run.

3. **Strengthen evidence beyond agent agreement**
   Next: directed + metamorphic tests, coverage feedback, bounded formal checks, and executable cycle-accurate Python references.